

Incidents Master Analysis

Limitations

- Missing values were present in several variables, particularly downtime-related observations.
- Industry classifications were represented using NAICS codes and may require additional mapping for business interpretation.
- The analysis focused on descriptive statistics and exploratory data analysis; causal relationships were not evaluated.
- Downtime and data exposure analyses were conducted using available observations only.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('incidents_master.csv')

incident_id = df['incident_id']
revenue = df['company_revenue_usd']
attack_vector = df['attack_vector_primary']
downtime_hours = df['downtime_hours']
```

Descriptive Analysis

Information of Dataset

```
print(df.info())

<class 'pandas.DataFrame'>
RangeIndex: 850 entries, 0 to 849
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   incident_id                           850 non-null    str
1   company_name                           850 non-null    str
2   company_revenue_usd                   850 non-null    float64
3   country_hq                             850 non-null    str
4   industry_primary                       850 non-null    str
5   industry_secondary                     153 non-null    str
6   employee_count                         850 non-null    int64
7   is_public_company                     850 non-null    bool
8   stock_ticker                           412 non-null    str
9   incident_date                           850 non-null    str
10  incident_date_estimated                 850 non-null    bool
11  discovery_date                           850 non-null    str
12  disclosure_date                           850 non-null    str
13  attack_vector_primary                   850 non-null    str
14  attack_vector_secondary                 211 non-null    str
15  attack_chain                           575 non-null    str
16  attributed_group                        482 non-null    str
17  attribution_confidence                  482 non-null    str
18  data_compromised_records                602 non-null    float64
19  data_type                               602 non-null    str
20  systems_affected                       850 non-null    str
21  downtime_hours                         420 non-null    float64
22  data_source_primary                     850 non-null    str
23  data_source_secondary                   386 non-null    str
24  data_source_type                        850 non-null    str
25  confidence_tier                         850 non-null    int64
26  quality_score                           850 non-null    float64
27  quality_grade                           850 non-null    str
28  review_flag                             70 non-null     str
29  notes                                   214 non-null    str
30  created_at                             850 non-null    str
31  updated_at                             850 non-null    str
dtypes: bool(2), float64(4), int64(2), str(24)
memory usage: 201.0 KB
None
```

- This contains all the initial information about the incidents masters dataset
- Company details, attack chains, disclosure dates, and quality grades are included within this dataset

Describing all numerical data points

```
print(df.describe())
```

```
count      company_revenue_usd  employee_count  data_compromised_records  \
mean      1.031337e+10          5.433418e+04          2.708462e+06
std       2.106158e+10          1.266599e+05          2.978069e+07
min       2.424181e+07          6.700000e+01          1.000000e+03
25%       2.327545e+08          1.087000e+03          1.200500e+04
50%       1.261375e+09          6.219500e+03          5.613300e+04
75%       8.692943e+09          4.359475e+04          2.891170e+05
max       1.488980e+11          1.411332e+06          5.497365e+08

count      downtime_hours  confidence_tier  quality_score
mean      107.208762        2.215294        79.963047
std       184.645681        1.202472        12.209540
min       1.830000          1.000000        50.040000
25%       25.822500         1.000000        71.727500
50%       53.600000         2.000000        80.815000
75%       119.800000        3.000000        90.072500
max       1951.620000        4.000000        99.780000
```

- This describes the descriptive statistics of all numerical data included within the dataset.
- With this function includes:
 - count : the number of valid/non-null data points; assess the completeness of the dataset by showing how many entries are valid.
 - mean : the average
 - std : the standard deviation
 - min : the minimum/lowest value
 - 25%. 50%, 75% : the quartiles (Q1, Q2 (median), Q3 respectively)
 - max : the maximum/highest value

```
pd.set_option('display.max_rows', None)
```

- Only to display all rows possible

Number of Attack Vectors

```
# Number of Attack Vectors
attack_vectors_numbers = attack_vector.value_counts()
print(attack_vectors_numbers)
```

```
attack_vector_primary
ransomware      228
phishing        147
data_breach     123
apt              84
malware         79
ddos            67
supply_chain    58
trojan          34
backdoor        30
Name: count, dtype: int64
```

- This contains all numbers of attack vectors within the dataset
- According to the data shown, Ransomware occurred much greater within the 850 incidents, summing up to 228 attacks, with backdoor having the lowest occurrence of 30

Company Type

```
company_type = df['is_public_company'].replace({True: "Public", False: "Private"}).value_counts()
print(company_type)
```

```
is_public_company
Private      438
Public       412
Name: count, dtype: int64
```

- This data shows that out of 850 companies within the dataset, 412 are within the public sector.

Downtime Hours

```
# Average Downtime Hours
print(downtime_hours.describe())
print(downtime_hours.skew())

count      420.000000
mean       107.208762
std        184.645681
min         1.830000
25%        25.822500
50%        53.600000
75%       119.800000
max       1951.620000
Name: downtime_hours, dtype: float64
5.744713468542288
```

- This data describes the central tendency; mean, median, and standard deviation is shown.
 - According to the results, the median is significantly lower than the mean downtime, which indicates a right-skewed distribution caused by small number of incidents (420 out of 850 documented incidents)
-

Records of Data Compromised

```
# Data Compromised Records
data_com_rec = df['data_compromised_records'].describe()
print(data_com_rec)

count      6.020000e+02
mean       2.708462e+06
std        2.978069e+07
min        1.000000e+03
25%        1.200500e+04
50%        5.613300e+04
75%        2.891170e+05
max        5.497365e+08
Name: data_compromised_records, dtype: float64
```

- This result describes the records of data compromised during the incidents. This may include the number of personal details or information that are leaked.
 - The results are also a right-skewed distribution.
-

Number of Quality Grades between Companies

```
# Quality Grade
quality_grade = df['quality_grade'].value_counts()

print(quality_grade)

quality_grade
Silver      382
Gold        311
Bronze      157
Name: count, dtype: int64
```

Attack Revenue + Downtime Analysis

```
# Attack Vector + Downtime Analysis
attack_downtime = df.groupby("attack_vector_primary")["downtime_hours"].agg([
    "count",
    "median",
    "mean",
])

print(attack_downtime)

      count  median  mean
attack_vector_primary
apt         75   59.270 109.240800
backdoor     0    NaN    NaN
data_breach  0    NaN    NaN
```

ddos	50	47.005	116.246000
malware	64	49.840	106.037500
phishing	0	NaN	NaN
ransomware	188	51.720	97.863191
supply_chain	43	62.930	135.759070
trojan	0	NaN	NaN

- This section analyzes the total count, median, and mean of downtime hours of each attack.
- The NaN value implies an unobserved downtime hours.

Attack Vectors + Data Compromised Analysis

```
# Attack Vector + Data Compromised Records Analysis
print("\nAttack Vector + Data Compromised Analysis")
record_compromised = df.groupby("attack_vector_primary")["data_compromised_records"].agg([
    "count",
    "mean",
    "median",
    "skew"
]).sort_values(by='median')

print(record_compromised)
```

Attack Vector + Data Compromised Analysis				
	count	mean	median	skew
ransomware	201	3.697027e+06	46614.0	13.910212
data_breach	102	1.613736e+06	54249.5	7.327722
phishing	125	4.592842e+06	58015.0	11.078428
malware	71	1.160265e+06	65421.0	4.138481
apt	75	6.005178e+05	68400.0	3.085297
trojan	28	7.595563e+05	83445.5	4.348966
backdoor	0	NaN	NaN	NaN
ddos	0	NaN	NaN	NaN
supply_chain	0	NaN	NaN	NaN

- This section provides the count, mean, median, and skew of data compromised on their respective attack vectors, with ransomware, phishing, and data breaches has the highest number of observed damage.
- NaN implies unobserved records.

Industry Sectors affected

```
# Industries affected
print("By downtime hours")
industries = df['industry_primary'].value_counts()
print(industries)
industries_downtime = df.groupby('industry_primary')['downtime_hours'].agg([
    "count",
    "mean",
    "median"
]).sort_values(by='count', ascending=False)
print(industries_downtime)

# -----
print("\nBy data compromised")

industries_records = df.groupby("industry_primary")['data_compromised_records'].agg([
    "count",
    "mean",
    "median"
]).sort_values(by='count', ascending=False)

industries_records = industries_records[industries_records["count"] >= 20]

print(industries_records)
```

By downtime hours	
industry_primary	
62	142
52	128
51	127
44-45	81
31-33	79
92	52
22	38
61	37
54	36

```

48-49    23
21       21
56       16
72       16
71       14
23       13
42       10
53        8
11        4
81        3
55        2
Name: count, dtype: int64

```

	count	mean	median
industry_primary			
62	76	87.485000	39.570
51	62	91.811935	51.250
52	58	129.959483	44.915
44-45	40	115.356500	44.670
31-33	39	116.253333	63.260
92	25	128.343200	122.910
61	21	155.570952	84.340
54	18	66.738889	39.245
22	17	92.385294	79.180
48-49	11	72.620000	55.710
23	10	69.462000	38.125
21	10	61.730000	39.960
56	8	86.328750	46.920
71	7	83.645714	81.750
72	6	75.080000	37.180
42	5	80.522000	50.220
53	4	552.572500	98.045
55	1	166.960000	166.960
81	1	51.790000	51.790
11	1	84.250000	84.250

```

By data compromised

```

	count	mean	median
industry_primary			
62	104	1.463184e+06	53962.0
52	97	7.350630e+05	55354.0
51	91	7.234125e+05	44832.0
44-45	52	2.784303e+06	82760.0
31-33	48	1.246162e+07	63753.5
92	36	4.802569e+05	55874.0
54	30	5.407680e+05	92072.5
22	29	1.650059e+07	122520.0
61	26	2.446100e+06	24600.5

- This section shows the industries affected at the attacks, their downtime hours, and the data compromised for each.
- According to the NAICS industry code, 62 (Health Care and Social Assistance) results to a more occurrence of incidents
- Industry 81 exhibited the highest median number of compromised records among industries with available observations; however, the finding is based on only three incidents.

Number of Missing Values

```

print("\nMissing Values")
print("-----")

missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)

missing_df = pd.DataFrame({
    "Missing Count": missing,
    "Missing %": missing_pct
})

print(missing_df.sort_values("Missing %", ascending=False))

```

```

Missing Values
-----

```

	Missing Count	Missing %
review_flag	780	91.76
industry_secondary	697	82.00
attack_vector_secondary	639	75.18
notes	636	74.82
data_source_secondary	464	54.59
stock_ticker	438	51.53
downtime_hours	430	50.59
attributed_group	368	43.29
attribution_confidence	368	43.29
attack_chain	275	32.35
data_compromised_records	248	29.18
data_type	248	29.18
data_source_type	0	0.00
data_source_primary	0	0.00
quality_score	0	0.00

quality_grade	0	0.00
systems_affected	0	0.00
created_at	0	0.00
confidence_tier	0	0.00
incident_id	0	0.00
company_name	0	0.00
attack_vector_primary	0	0.00
disclosure_date	0	0.00
discovery_date	0	0.00
incident_date_estimated	0	0.00
incident_date	0	0.00
is_public_company	0	0.00
employee_count	0	0.00
industry_primary	0	0.00
country_hq	0	0.00
company_revenue_usd	0	0.00
updated_at	0	0.00

- This output checks all missing values between columns, showing the number of cells within the spreadsheet that contains no value.

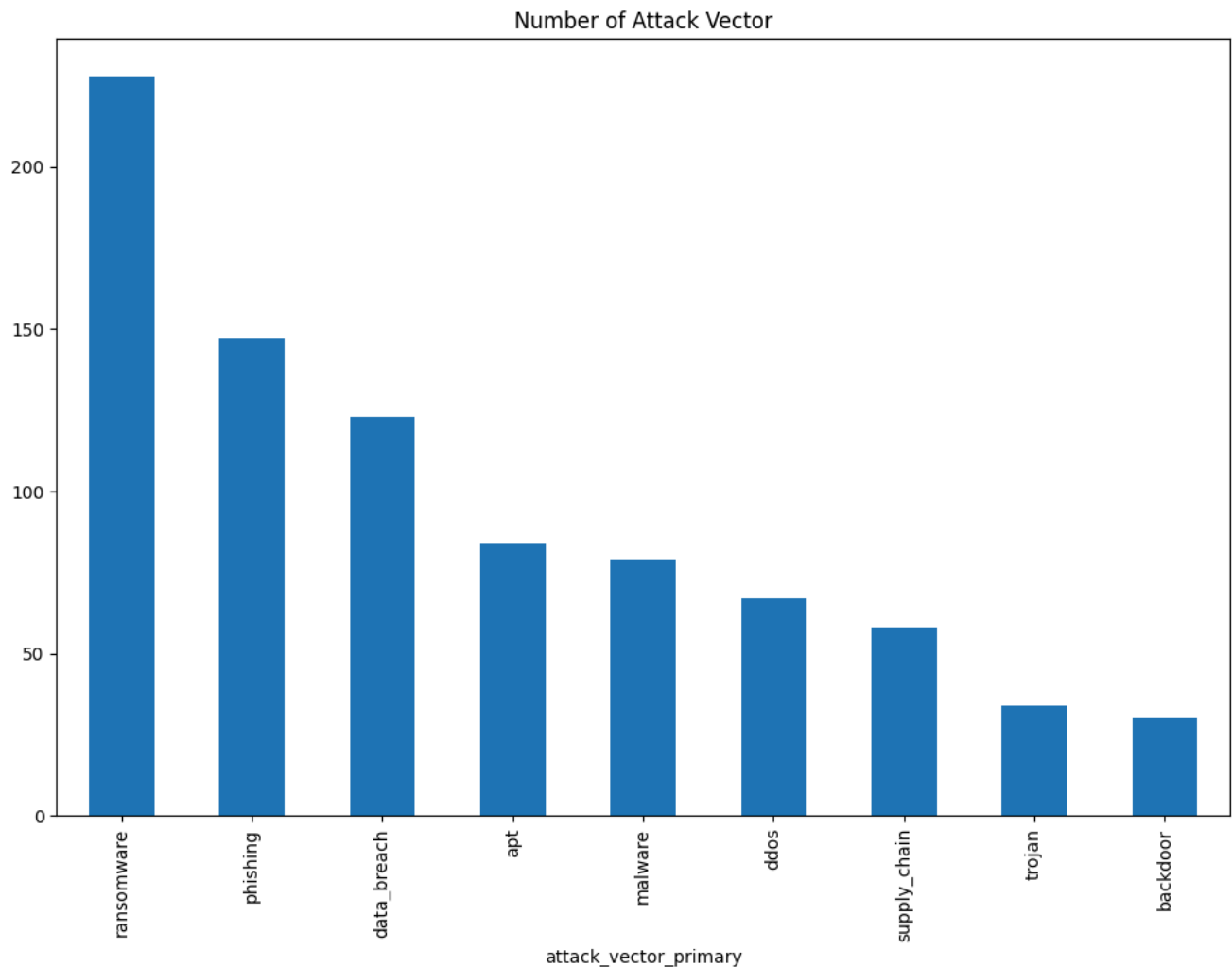
Graphs

Number of Attack Vectors

```
# Number of Attack Vectors
attack_vectors_numbers = attack_vector.value_counts()

plt.figure(figsize=(12,8))
attack_vector_plots = attack_vectors_numbers.plot(kind='bar')
plt.title("Number of Attack Vector")
```

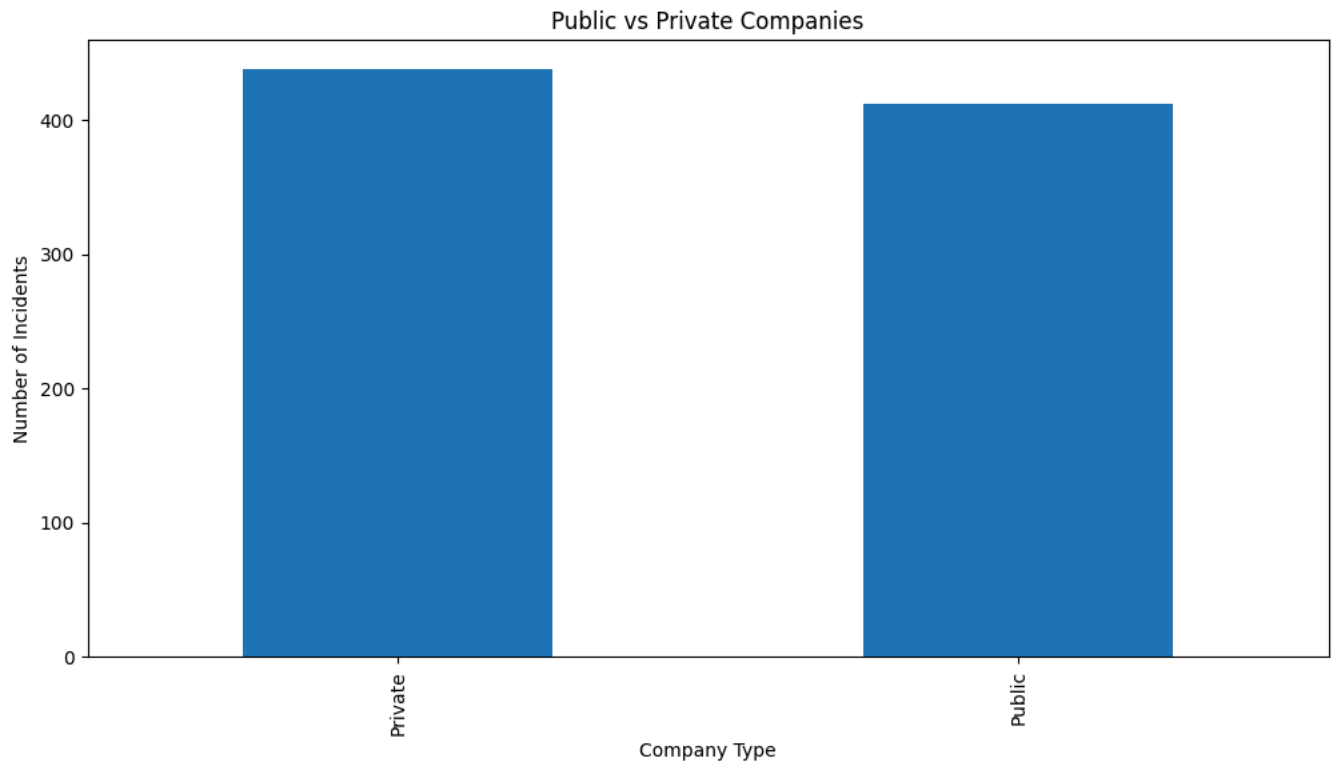
```
Text(0.5, 1.0, 'Number of Attack Vector')
```



Number of Public and Private Companies

```
# Public/Private Companies
company_type = (
    df["is_public_company"]
    .replace({True: "Public", False: "Private"})
    .value_counts()
)
plt.figure(figsize=(12,6))
company_type.plot(kind="bar")

plt.title("Public vs Private Companies")
plt.xlabel("Company Type")
plt.ylabel("Number of Incidents")
plt.show()
```



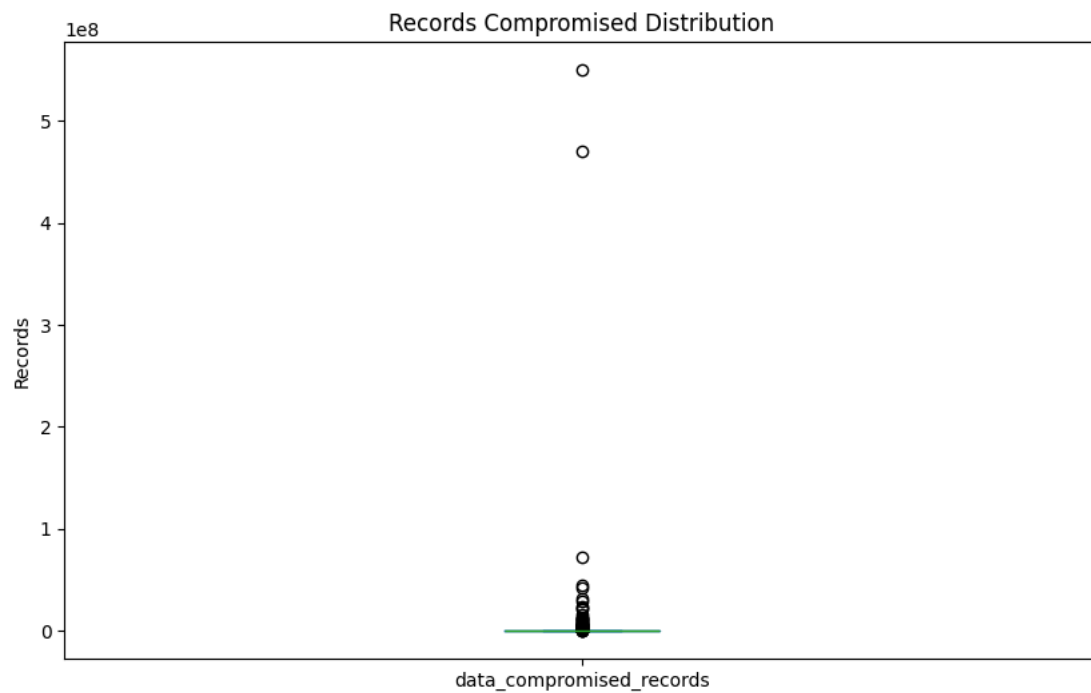
Records of Data Compromised

```
# Data Compromised Records
data_com_rec = df['data_compromised_records']
plt.figure(figsize=(10,6))

df["data_compromised_records"].dropna().plot(kind='box')

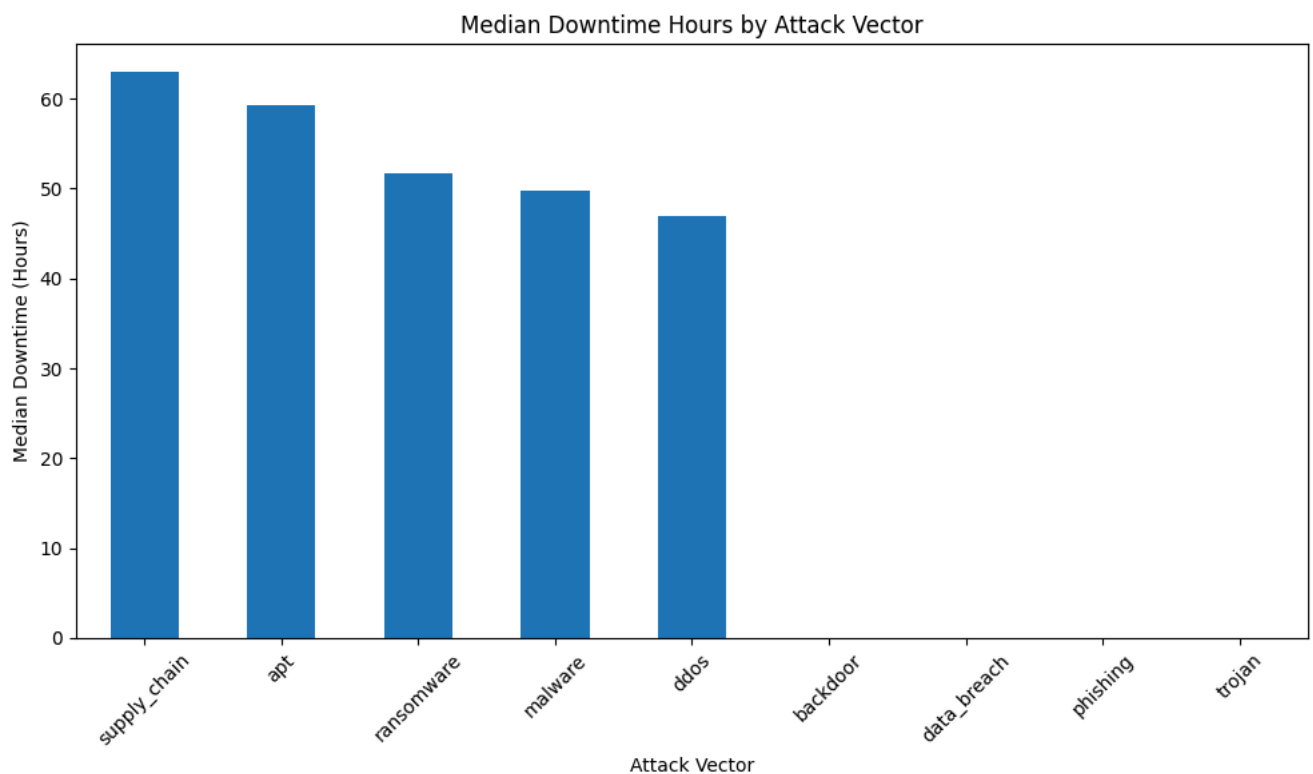
plt.title("Records Compromised Distribution")
plt.ylabel("Records")

plt.show()
```



Downtime Hours by Attack Vectors

```
attack_downtime = (  
    df.groupby("attack_vector_primary")["downtime_hours"]  
    .median()  
    .sort_values(ascending=False)  
)  
  
plt.figure(figsize=(10,6))  
  
attack_downtime.plot(kind="bar")  
  
plt.title("Median Downtime Hours by Attack Vector")  
plt.xlabel("Attack Vector")  
plt.ylabel("Median Downtime (Hours)")  
  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```



Note that backdoor, data breach, phishing, and trojan have undocumented downtime hours

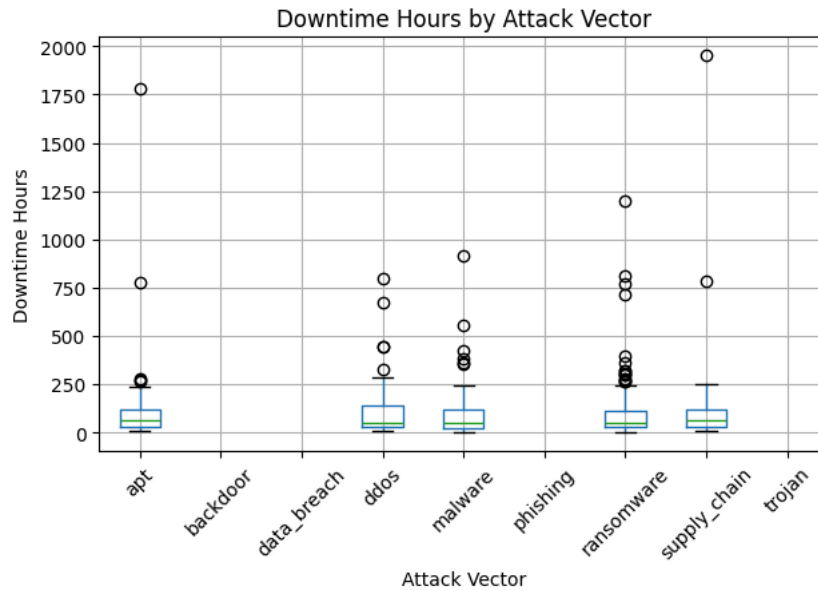

```
plt.figure(figsize=(12,6))

df.boxplot(
    column="downtime_hours",
    by="attack_vector_primary",
    rot=45
)

plt.title("Downtime Hours by Attack Vector")
plt.suptitle("")
plt.xlabel("Attack Vector")
plt.ylabel("Downtime Hours")

plt.tight_layout()
plt.show()
```

<Figure size 1200x600 with 0 Axes>



Data Compromised Records by Attack Vector

```
records_by_attack = (
    df.groupby("attack_vector_primary")["data_compromised_records"]
      .agg(["count", "mean", "median"])
      .sort_values("median", ascending=False)
)

records_median = (
    df.groupby("attack_vector_primary")["data_compromised_records"]
      .median()
      .sort_values(ascending=False)
)

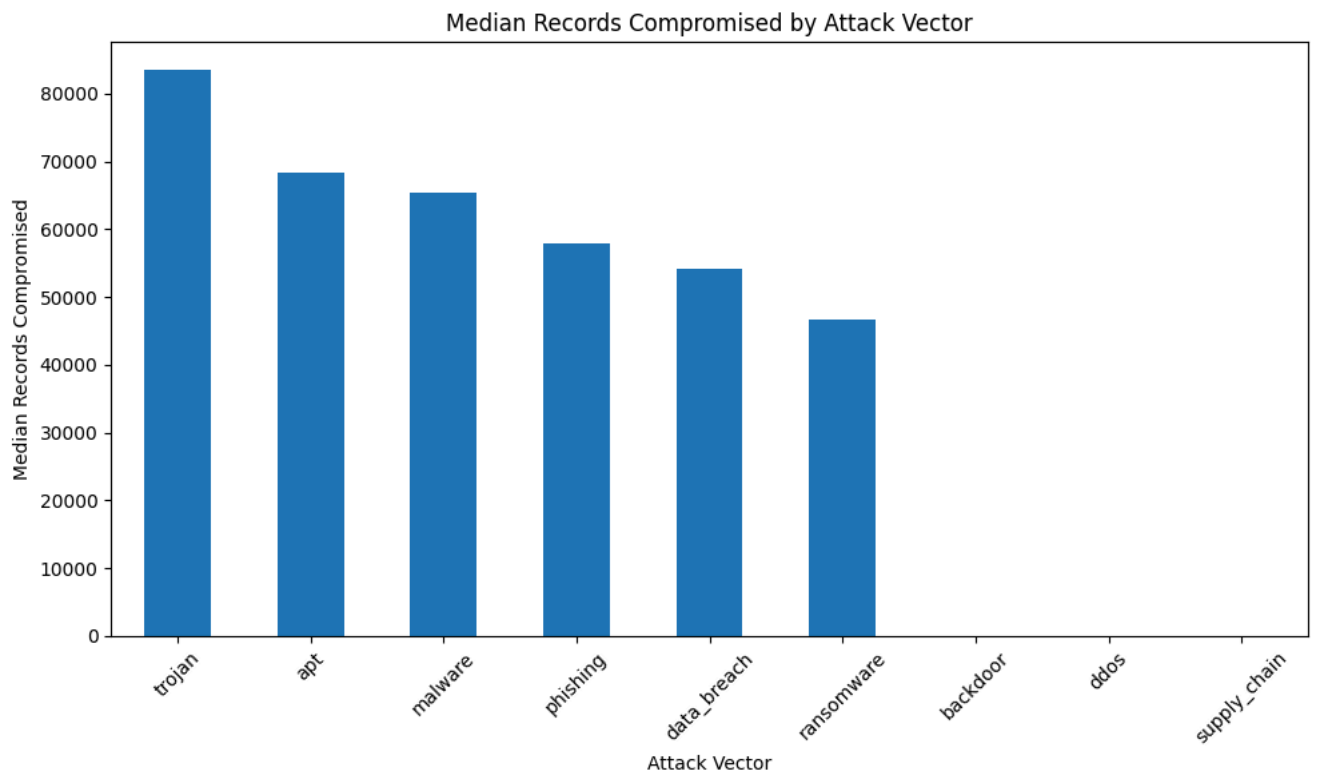
plt.figure(figsize=(10,6))

records_median.plot(kind="bar")

plt.title("Median Records Compromised by Attack Vector")
plt.xlabel("Attack Vector")
plt.ylabel("Median Records Compromised")

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```



Downtime Hours by Industry Sectors

```
industry_median = (
    df.groupby("industry_primary")["downtime_hours"]
      .median()
      .sort_values(ascending=False)
)

plt.figure(figsize=(10,6))

industry_median.plot(kind="bar")

plt.title("Median Downtime Hours by Industry")
plt.xlabel("Industry")
plt.ylabel("Median Downtime (Hours)")

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```

```
# Median Records Compromised by Industry

industry_records = (
    df.groupby("industry_primary")["data_compromised_records"]
      .median()
      .sort_values(ascending=False)
)

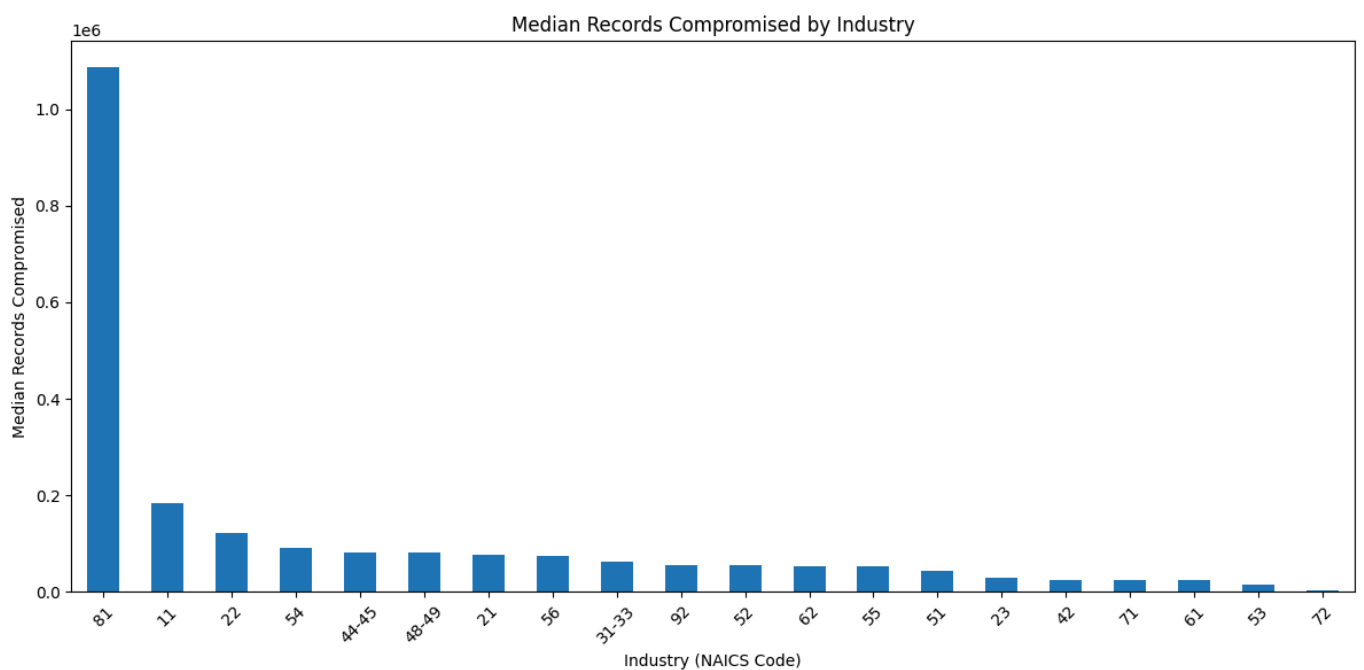
plt.figure(figsize=(12,6))

industry_records.plot(
    kind="bar"
)

plt.title("Median Records Compromised by Industry")
plt.xlabel("Industry (NAICS Code)")
plt.ylabel("Median Records Compromised")

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```



Findings